

TICKET BOOKING **MANAGEMENT SYSTEM** **(TBMS)**

SUBMITTED BY:-

- | | | |
|-------------------|----------------|-------|
| 1) NAYAN RAJ | (GROUP LEADER) | _____ |
| 2) AMIT KUMAR | (GROUP MEMBER) | _____ |
| 3) SATISH KUMAR | (GROUP MEMBER) | _____ |
| 4) VIVEK KUMAR | (GROUP MEMBER) | _____ |
| 5) RAJ KUMAR RANA | (GROUP MEMBER) | _____ |
| 6) AJANISH KUMAR | (GROUP MEMBER) | _____ |

GUIDED AND SUPERVISED BY :-

(prof) DR.BINOD KUMAR Signature: _____

BCA 4th SEMESTER

SESSION:-2024-2027

AISECT UNIVERSITY HAZARIBAGH (JHARKHAND)

1. OBJECTIVE :-

1. To automate the bus booking process

- The main objective of this project is to eliminate manual booking and provide a simple automated system using shell scripting.

2. To manage bus details efficiently

- The system stores and handles all bus-related information such as bus number, route, timing, category, price, and seat availability.

3. To provide easy booking functionality

- Users can book tickets by selecting source, destination, bus, category, and number of seats in a structured way.

2. FEATURES :-

1. Bus Management:-

- Add new bus details
- View all buses
- Search buses based on route
- Delete bus records

2. Booking Management:-

- Create new booking
- Input customer details (name, phone)
- Select bus and category (AC / Non-AC / Sleeper)
- Automatic fare calculation
- Seat availability check

3. Ticket Generation:-

- Generates ticket after booking
- Displays booking details clearly
- Shows seat numbers and total amount

4. Payment System:-

- Validates booking before payment
- Displays total payable amount
- Supports multiple payment methods (UPI / Card / Cash)
- Prevents duplicate payments

5. Cancellation Module:-

- Cancel booking using Booking ID
- Restores seat availability
- Deletes booking and payment records
- Stores cancellation history

6. File-Based Storage System:-

- bus.txt → stores bus data
- booking.txt → stores booking records
- payment.txt → stores payment details
- cancellation.txt → stores cancellation data

3. SYSTEM REQUIREMENT:-

➤ Hardware Required:

- Standard PC/Laptop (any modern system)
- Processor: Minimum dual-core CPU
- RAM: 4 GB or higher
- Storage: At least 1 GB free disk space (for scripts, text files, and logs)
- Input/Output: Keyboard and display terminal (CLI-based interaction)

➤ Software Required:

- Operating System: Linux/Unix environment (or Windows with WSL)
- Shell: POSIX-compliant shell (/bin/sh)
- Utilities: awk, grep, cut, mv, printf, read (all standard in Unix/Linux)
- Text Editor: Any editor (e.g., vim, nano) for modifying scripts and data files
- File System: Flat-file storage using .txt files for buses, bookings, payments, and cancellations
- Documentation :- Microsoft Word

4. ENTITY SET:-

ENTITY	ATTRIBUTES	OPERATION
BUS	BusNo (Primary Key), BusName, Source, Destination, DepartureTime, ArrivalTime, Category, Price, TotalSeats, AvailableSeats	Add Bus, View All Buses, Search Bus (by Source & Destination), Delete Bus, Update Seat Availability
BOOKING	BookingID (Primary Key), Name, PhoneNumber, BusNo (Foreign Key) Category, SeatsBooked, TotalAmount	Create Booking, View Booking Details, Validate Booking, Generate Ticket, Calculate Total Amount

5. REQUIRED TABLES:-

➤ **Bus Table:-**

- **Columns:** Bus ID, Bus Name, Bus Number, Source City, Destination City, Departure Time, Arrival Time, Category, Ticket Price, Total Seats, Available Seats
- **File:** bus.txt

➤ **Booking Table:-**

- **Columns:** Booking ID, Customer Name, Phone Number, Bus Number, Category, Seats Booked, Total Amount
- **File:** booking.txt

➤ **Payment Table:-**

- **Columns:** Payment ID, Booking ID, Amount, Payment Method, Status
- **File:** payment.txt

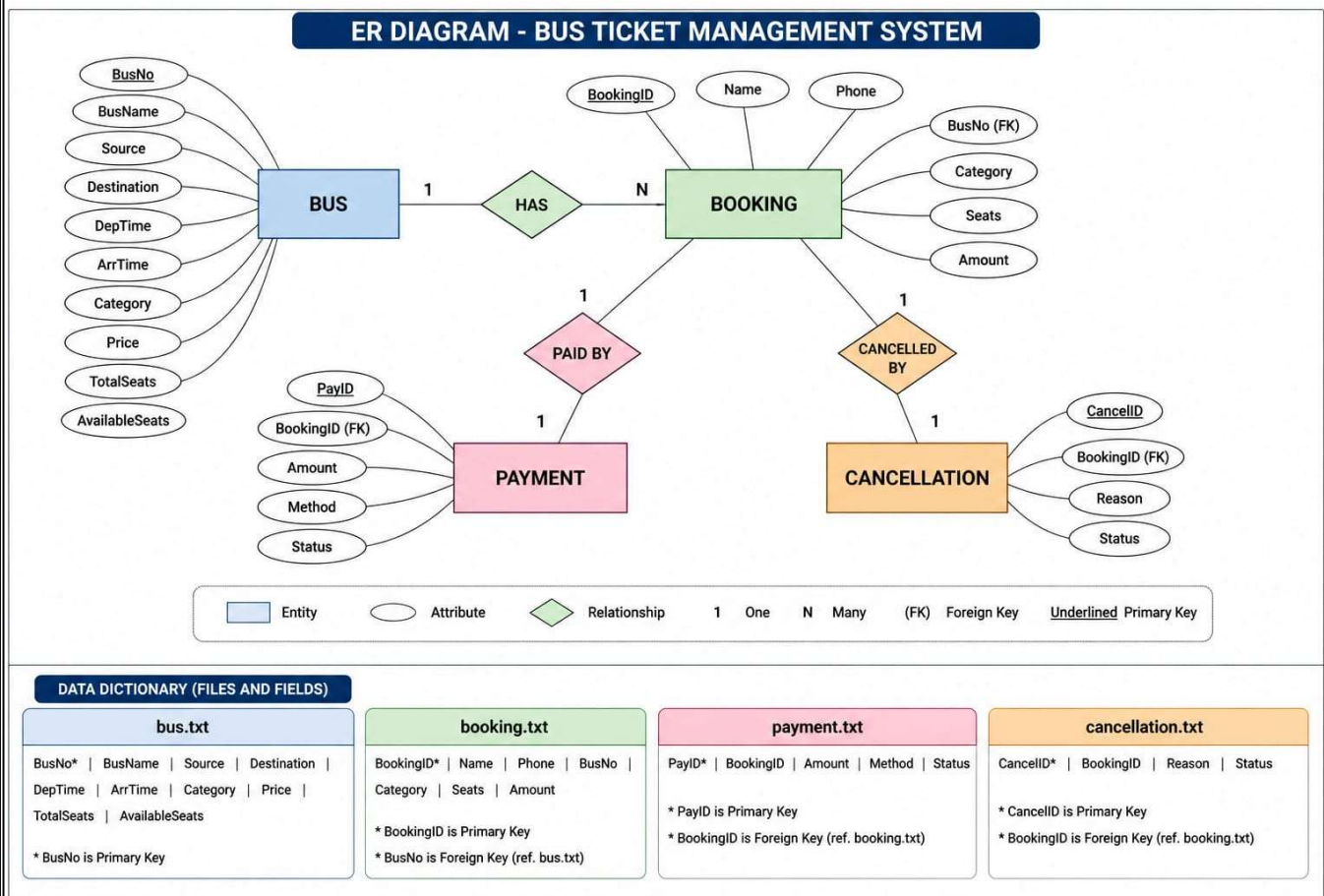
➤ **Cancellation Table:-**

- **Columns:** Cancellation ID, Booking ID, Reason, Status
- **File:** cancellation.txt

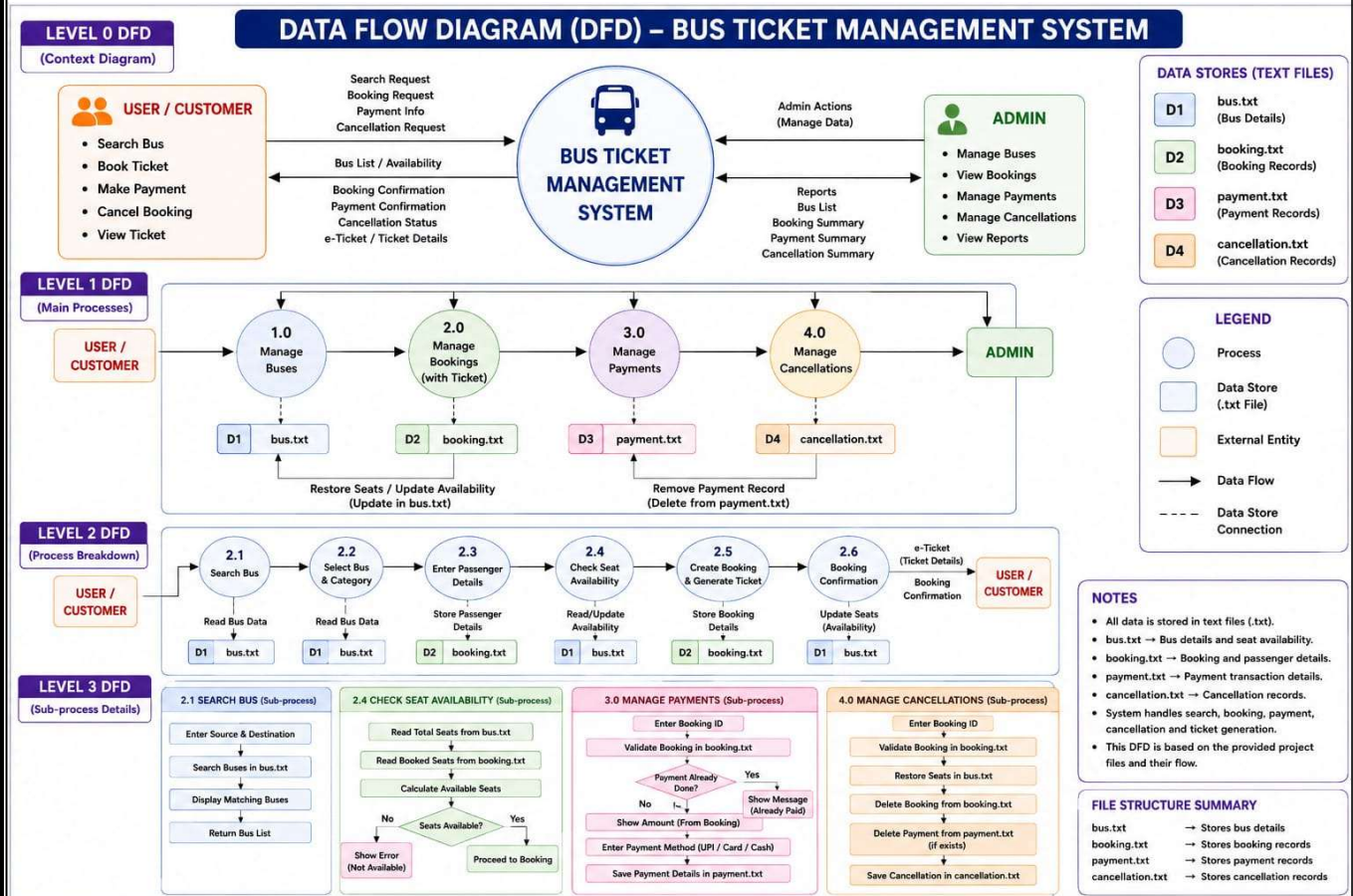
6. RELATIONSHIP BETWEEN TABLES:-

Relationships:

- Bus ↔ Booking (1:N)
- Booking ↔ Payment (1:1)
- Booking ↔ Cancellation (1:1, optional)



7. DATA FLOW DIAGRAM(DFD):-



8. SOFTWARE MODULES:-

➤ MODULE 1:- MAIN MENU MODULE

Description: This module acts as the central control of the system.

Working:

- Displays menu options:
- Bus Management
- Booking Management
- Payment Management
- Cancellation Management
- Based on user input, it redirects to the respective module.

➤ Module 2:- BUS MANAGEMENT

Description: This module handles all operations related to bus details.

Working:

- The user can add new bus records by entering bus name, number, route, timing, category, price, and seat details.
- Users can view all buses in a structured format.
- The system allows searching buses based on source and destination (case-insensitive).

➤ MODULE 3:- BOOKING MODULE (WITH TICKET)

Description: This module manages ticket booking and generates ticket details.

Working:

- The user enters booking ID, name, phone number, source, and destination.
- The system searches available buses from bus.txt.
- The user selects bus number and category.
- The system checks seat availability.
- If seats are available: It calculates total amount (price × seats).
- Updates available seats in bus.txt.
- After booking, the system generates a ticket showing all details.

➤ **MODULE 4:- PAYMENT MODULE**

Description: This module handles payment processing for bookings.

Working:

- The user enters payment ID and booking ID.
- The system validates whether the booking exists.
- It checks if payment is already done (prevents duplicate payment).
- Fetches booking details and displays total amount.
- The user confirms payment and selects payment method.

➤ **MODULE 5:- CANCELLATION MODULE**

Description: This module manages booking cancellations.

Working:

- The user enters booking ID and cancellation ID.
- The system validates the booking record.
- It restores the booked seats back to bus.txt.
- Deletes the booking record from booking.txt.
- Deletes associated payment record from payment.txt

9. CODES:-

➤ MODULE 1:- MAIN MENU MODULE

```
while true
do
echo "=====
echo "    BUS BOOKING MANAGEMENT SYSTEM"
echo "=====
echo "1. Manage Buses"
echo "2. Manage Bookings (with Tickets)"
echo "3. Manage Payments"
echo "4. Manage Cancellations"
echo "5. Exit"
echo "-----"
printf "Please enter your choice: "
read ch

case $ch in
1) echo "Opening Bus Management Module..."
sh bus.sh
;;

2)echo "Opening Booking & Ticket Module..."
sh booking.sh
;;

3)echo "Opening Payment Module..."
sh payment.sh
;;

4)echo "Opening Cancellation Module..."
sh cancellation.sh
;;

5)echo "=====
echo "Thank you for using the Bus Booking Management System."
echo "Session has been closed successfully."
echo "=====
exit ;;
*)echo "Invalid choice. Please try again."
;;

esac

done
```

➤ MODULE 2:- BUS MANAGEMENT

```
file="bus.txt"; touch $file
while true
do
echo "=====
echo "          BUS MANAGEMENT SYSTEM"
echo "=====
echo "1. Add New Bus"
echo "2. View All Buses"
echo "3. Search Bus by Route"
echo "4. Delete Bus"
echo "5. Back to Main Menu"
echo "-----"
printf "Please enter your choice: "
read ch
case $ch in

1)printf "Enter Bus Name: "
read name
printf "Enter Bus Number (e.g., BUS-101): "
read busno
# Duplicate check
check=$(awk -F'|' -v b="$busno" '$2==b' $file)
if [ ! -z "$check" ]; then
    echo "This Bus Number already exists. Please use a unique Bus Number."
    continue
fi
printf "Enter Source City: "
readsrc
printf "Enter Destination City: "
read dest
printf "Enter Departure Time (e.g., 10:00AM): "
read dtime
printf "Enter Arrival Time (e.g., 03:00PM): "
read atime
printf "Enter Category (AC / Non-AC / Sleeper): "
read cat
printf "Enter Ticket Price: "
read price
printf "Enter Total Seats: "
read total
avail=$total
echo "$name|$busno|$src|$dest|$dtime|$atime|$cat|$price|$total|$avail" >> $file

echo "Bus has been added successfully with all details." ;;
```

```

2)if [ ! -s $file ]; then
    echo "No bus records are available at the moment."
else
    printf "%-12s | %-8s | %-10s | %-12s | %-8s | %-8s | %-10s | %-6s | %-6s | %-6s\n" \
    "Name" "BusNo" "Source" "Destination" "Dep" "Arr" "Category" "Price" "Total" "Avail"
    echo "-----"
    while IFS='|' read a b c d e f g h i j
    do
        [ -z "$a" ] && continue
        printf "%-12s | %-8s | %-10s | %-12s | %-8s | %-8s | %-10s | %-6s | %-6s | %-6s\n" \
        "$a" "$b" "$c" "$d" "$e" "$f" "$g" "$h" "$i" "$j"
    done < $file
fi ;;
3)printf "Enter Source City: "
read src
printf "Enter Destination City: "
read dest
res=$(awk -F'|' -v s="$src" -v d="$dest" '
tolower($3)==tolower(s) && tolower($4)==tolower(d)' $file)
if [ -z "$res" ]; then
    echo "No buses found for the given route."
else
    echo "Matching buses for your selected route are:"
    printf "%-12s | %-8s | %-10s | %-12s | %-8s | %-8s | %-10s | %-6s | %-6s | %-6s\n" \
    "Name" "BusNo" "Source" "Destination" "Dep" "Arr" "Category" "Price" "Total" "Avail"
    echo "-----"
    echo "$res" | while IFS='|' read a b c d e f g h i j
    do
        printf "%-12s | %-8s | %-10s | %-12s | %-8s | %-8s | %-10s | %-6s | %-6s | %-6s\n" \
        "$a" "$b" "$c" "$d" "$e" "$f" "$g" "$h" "$i" "$j"
    done
fi
;;
4)printf "Enter Bus Number to delete: "
read id
res=$(awk -F'|' -v b="$id" ' $2==b' $file)
if [ -z "$res" ]; then
    echo "No matching bus record was found."
else
    awk -F'|' -v b="$id" ' $2!=b' $file > temp && mv temp $file
    echo "Bus record has been deleted successfully."
fi ;;
5) break ;;
*)echo "Invalid choice. Please try again." ;;
esac
done

```

➤ **MODULE 3:- BOOKING MODULE (WITH TICKET)**

```
file="booking.txt"; touch $file
bus="bus.txt"
while true
do
echo "=====
echo "          BOOKING MANAGEMENT SYSTEM"
echo "=====
echo "1. Create New Booking"
echo "2. View All Bookings"
echo "3. Delete Booking"
echo "4. Search Booking"
echo "5. View Ticket"
echo "6. Back to Main Menu"
echo "-----"
printf "Please enter your choice: "
read ch

case $ch in
# ----- CREATE BOOKING -----
1)
printf "Enter Booking ID (e.g., BKG-1): "
read bid
check=$(grep "^$bid|" $file)
if [ ! -z "$check" ]; then
    echo "This Booking ID already exists. Please use a different ID."
    continue
fi
printf "Enter Customer Name: "
read name
printf "Enter Phone Number: "
read phone
printf "Enter Source City: "
read src
printf "Enter Destination City: "
read dest

# Show buses
res=$(awk -F'|' -v s="$src" -v d="$dest" '
tolower($3)==tolower(s) && tolower($4)==tolower(d)' $bus)
if [ -z "$res" ]; then
    echo "No buses are available for the selected route."
    continue
fi
echo "Available buses for your route:"
```

```

printf "%-10s | %-8s | %-10s | %-10s | %-8s | %-8s | %-10s | %-6s | %-6s\n" \
"Name" "BusNo" "Source" "Destination" "Dep" "Arr" "Category" "Price" "Avail"
echo "-----"
echo "$res" | while IFS='|' read a b c d e f g h i j
do
    printf "%-10s | %-8s | %-10s | %-10s | %-8s | %-8s | %-10s | %-6s | %-6s\n" \
        "$a" "$b" "$c" "$d" "$e" "$f" "$g" "$h" "$i" "$j"
done
printf "Enter Bus Number: "
read busno
printf "Enter Category (AC / Non-AC / Sleeper): "
read cat
line=$(awk -F'|' -v b="$busno" -v c="$cat" 'tolower($2)==tolower(b) && tolower($7)==tolower(c)' $bus

if [ -z "$line" ]; then
    echo "Invalid Bus Number or Category."
    continue
fi
price=$(echo $line | cut -d'|' -f8)
avail=$(echo $line | cut -d'|' -f10)
printf "Enter number of seats to book: "
read qty

if [ "$qty" -gt "$avail" ]; then
    echo "Only $avail seats are available. Please try again."
    continue
fi

total=$((price * qty))
# Save booking
echo "$bid|$name|$phone|$busno|$cat|$qty|$total" >> $file

# Update seats
new_avail=$((avail - qty))

awk -F'|' -v b="$busno" -v c="$cat" -v new="$new_avail" 'BEGIN{OFS="|"}'
{
    if (tolower($2)==tolower(b) && tolower($7)==tolower(c))
        $10=new
    print
}' $bus > temp && mv temp $bus
echo "-----"
echo "Booking has been completed successfully."
echo "Total amount for your booking is: ₹$total"
echo "-----"
;;

```

```

# ----- VIEW BOOKINGS -----
2)if [ ! -s $file ]; then
    echo "No booking records are available."
else
    printf "%-10s | %-12s | %-12s | %-8s | %-10s | %-5s | %-8s\n" \
    "BookID" "Name" "Phone" "BusNo" "Category" "Seats" "Amount"
    echo "-----"
    while IFS='|' read a b c d e f g
    do
        [ -z "$a" ] && continue
        printf "%-10s | %-12s | %-12s | %-8s | %-10s | %-5s | %-8s\n" \
        "$a" "$b" "$c" "$d" "$e" "$f" "$g"
    done < $file
fi
;;

# ----- DELETE BOOKING -----
3)printf "Enter Booking ID to delete: "
read id
line=$(grep "^$id|" $file)

if [ -z "$line" ]; then
    echo "No booking record found."
else
    busno=$(echo $line | cut -d'|' -f4)
    cat=$(echo $line | cut -d'|' -f5)
    qty=$(echo $line | cut -d'|' -f6)

    old=$(awk -F'|' -v b="$busno" -v c="$cat" 'tolower($2)==tolower(b) && tolower($7)==tolower(c){print $10}' $bus)
    new=$((old + qty))

    awk -F'|' -v b="$busno" -v c="$cat" -v new="$new" '
BEGIN{OFS="|"}
{
    if (tolower($2)==tolower(b) && tolower($7)==tolower(c))
        $10=new
    print
}' $bus > temp && mv temp $bus
    awk -F'|' -v id="$id" '$1!=id' $file > temp && mv temp $file
    echo "Booking has been deleted and seats have been restored."
fi
;;

# ----- SEARCH -----
4)printf "Enter Booking ID to search: "

```

```

read id
grep "^$id$" $file || echo "No booking found."
;;

# ----- TICKET -----
5)printf "Enter Booking ID to view ticket: "
read bid
res=$(grep "^$bid$" $file)
if [ -z "$res" ]; then
    echo "No booking record found."
else
    name=$(echo $res | cut -d'|' -f2)
    phone=$(echo $res | cut -d'|' -f3)
    busno=$(echo $res | cut -d'|' -f4)
    cat=$(echo $res | cut -d'|' -f5)
    qty=$(echo $res | cut -d'|' -f6)
    amount=$(echo $res | cut -d'|' -f7)
    echo "=====
    echo "          TICKET DETAILS"
    echo "=====
    echo "Booking ID : $bid"
    echo "Name      : $name"
    echo "Phone     : $phone"
    echo "Bus No    : $busno"
    echo "Category  : $cat"
    echo "Seats     : $qty"
    echo "Amount    : ₹$amount"
    echo "-----"
    i=1
    while [ $i -le $qty ]
    do
        echo "Ticket No : TKT-$bid-$i | Seat-$i"
        i=$((i+1))
    done
    echo "=====
fi
;;

6) break ;;
*)echo "Invalid choice. Please try again."
;;

esac
done

```

➤ MODULE 4:- PAYMENT MODULE

```
file="payment.txt"; touch $file
booking="booking.txt"
while true
do
echo "=====
echo "          PAYMENT MANAGEMENT SYSTEM"
echo "=====
echo "1. Make Payment"
echo "2. View All Payments"
echo "3. Back to Main Menu"
echo "-----"
printf "Please enter your choice: "
read ch
case $ch in

# ----- MAKE PAYMENT -----
1)printf "Enter Payment ID (e.g., PAY-1): "
read pid
# Duplicate Payment ID check
check=$(grep "^$pid$" $file)
if [ ! -z "$check" ]; then
    echo "This Payment ID already exists. Please use a different ID."
    continue
fi
printf "Enter Booking ID: "
read bid
# Check booking exists
bline=$(grep "^$bid$" $booking)
if [ -z "$bline" ]; then
    echo "Invalid Booking ID. Please enter a valid one."
    continue
fi
# Check if already paid
paid=$(awk -F'|' -v b="$bid" ' $2==b' $file)
if [ ! -z "$paid" ]; then
    echo "Payment has already been completed for this Booking ID."
    continue
fi
# Extract details
name=$(echo $bline | cut -d'|' -f2)
phone=$(echo $bline | cut -d'|' -f3)
busno=$(echo $bline | cut -d'|' -f4)
cat=$(echo $bline | cut -d'|' -f5)
qty=$(echo $bline | cut -d'|' -f6)
```



```

amount=$(echo $bline | cut -d'|' -f7)
# Show payment summary
echo "-----"
echo "Booking Details:"
echo "Booking ID : $bid"
echo "Name      : $name"
echo "Phone     : $phone"
echo "Bus No    : $busno"
echo "Category  : $cat"
echo "Seats     : $qty"
echo "Total Amount Payable: ₹$amount"
echo "-----"
printf "Do you want to proceed with the payment? (y/n): "
read confirm
if [ "$confirm" != "y" ]; then
    echo "Payment has been cancelled by the user."
    continue
fi
# Payment method
printf "Enter Payment Method (UPI / Card / Cash): "
read method
# Save payment
echo "$pid|$bid|$amount|$method|paid" >> $file
echo "-----"
echo "Payment has been completed successfully."
echo "-----"
;;

# ----- VIEW PAYMENTS -----
2)
if [ ! -s $file ]; then
    echo "No payment records are available."
else
    printf "%-10s | %-10s | %-10s | %-12s | %-10s\n" \ "PayID" "BookID" "Amount" "Method" "Status"
    echo "-----"
    while IFS='|' read a b c d e
    do
        [ -z "$a" ] && continue
        printf "%-10s | %-10s | %-10s | %-12s | %-10s\n" \ "$a" "$b" "$c" "$d" "$e"
    done < $file
fi ;;

3) break ;;
*)echo "Invalid choice. Please try again." ;;
esac
done

```

➤ MODULE 5:- CANCELLATION MODULE

```
file="cancellation.txt"; touch $file
booking="booking.txt"
bus="bus.txt"
payment="payment.txt"
while true
do
echo "=====
echo "      CANCELLATION MANAGEMENT SYSTEM"
echo "=====
echo "1. Cancel Booking"
echo "2. View All Cancellations"
echo "3. Back to Main Menu"
echo "-----"
printf "Please enter your choice: "
read ch
case $ch in

# ----- CANCEL BOOKING -----
1)printf "Enter Booking ID to cancel: "
read bid
# Check booking exists
bline=$(grep "^$bid$" $booking)
if [ -z "$bline" ]; then
    echo "Invalid Booking ID. No such booking exists."
    continue
fi
printf "Enter Cancellation ID (e.g., CAN-1): "
read cid
# Duplicate cancellation ID check
check=$(grep "^$cid$" $file)
if [ ! -z "$check" ]; then
    echo "This Cancellation ID already exists."
    continue
fi
printf "Enter Reason for Cancellation: "
read reason
# Extract booking details
busno=$(echo $bline | cut -d'|' -f4)
cat=$(echo $bline | cut -d'|' -f5)
qty=$(echo $bline | cut -d'|' -f6)

# Restore seats in bus.txt
line=$(awk -F'|' -v b="$busno" -v c="$cat" 'tolower($2)==tolower(b) && tolower($7)==tolower(c)
' $bus)
```

```

old=$(echo $line | cut -d'|' -f10)
new=$((old + qty))
awk -F'|' -v b="$busno" -v c="$cat" -v new="$new" '
BEGIN{OFS="|"}
{
    if (tolower($2)==tolower(b) && tolower($7)==tolower(c))
        $10=new
    print
}' $bus > temp && mv temp $bus
# Delete booking
awk -F'|' -v id="$bid" '$1!=id' $booking > temp && mv temp $booking
#Delete payment (important fix)
if [ -f "$payment" ]; then
    awk -F'|' -v id="$bid" '$2!=id' $payment > temp && mv temp $payment
fi

# Save cancellation
echo "$cid|$bid|$reason|cancelled" >> $file
echo "-----"
echo "Booking has been cancelled successfully."
echo "Seats have been restored and payment (if any) removed."
echo "-----"
;;

# ----- VIEW CANCELLATIONS -----
2)if [ ! -s $file ]; then
    echo "No cancellation records are available."
else
    printf "%-10s | %-10s | %-20s | %-10s\n" \ "CancelID" "BookID" "Reason" "Status"
    echo "-----"
    while IFS='|' read a b c d
    do
        [ -z "$a" ] && continue
        printf "%-10s | %-10s | %-20s | %-10s\n" \ "$a" "$b" "$c" "$d"
    done < $file
fi
;;

3) break ;;

*)echo "Invalid choice. Please try again." ;;
esac
done

```

10. INUPUT/OUTPUT:-

➤ Module 1:- BUS MANAGEMENT

```
=====
                        BUS MANAGEMENT SYSTEM
=====
1. Add New Bus
2. View All Buses
3. Search Bus by Route
4. Delete Bus
5. Back to Main Menu
-----
Please enter your choice: 1
Enter Bus Name: LAXMI
Enter Bus Number (e.g., BUS-101): BUS-105
Enter Source City: DHANBAD
Enter Destination City: DEOGARH
Enter Departure Time (e.g., 10:00AM): 3:00 PM
Enter Arrival Time (e.g., 03:00PM): 2:50 PM
Enter Category (AC / Non-AC / Sleeper): AC
Enter Ticket Price: 500
Enter Total Seats: 45
Bus has been added successfully with all details.
```

1. Add New Bus
2. View All Buses
3. Search Bus by Route
4. Delete Bus
5. Back to Main Menu

Please enter your choice: 2

Name	BusNo	Source	Destination	Dep	Arr	Category	Price	Total	Avail
Shivam	BUS-101	RANCHI	HAZARIBAGH	10:00AM	11:00AM	NON-AC	800	40	35
HEMKUNT	BUS-103	RANCHI	DEOGARH	12:00 PM	11:30 AM	AC	500	40	40
PAMMI	BUS-102	RANCHI	HAZARIBAGH	7:00 AM	6:30 AM	Sleeper	1000	40	40
VIJAY	BUS-104	HAZARIBAGH	DHANBAD	12:00 PM	11:00 AM	AC	750	40	40
LAXMI	BUS-105	DHANBAD	DEOGARH	3:00 PM	2:50 PM	AC	500	45	45

➤ MODULE 2:- BOOKING MODULE (WITH TICKET)

BOOKING MANAGEMENT SYSTEM

- ```
=====
1. Create New Booking
2. View All Bookings
3. Delete Booking
4. Search Booking
5. View Ticket
6. Back to Main Menu
=====
```

Please enter your choice: 1

Enter Booking ID (e.g., BKG-1): BKG-1

Enter Customer Name: NAYAN RAJ

Enter Phone Number: 9931871581

Enter Source City: DHANBAD

Enter Destination City: DEOGARH

Available buses for your route:

| Name  | BusNo   | Source  | Destination | Dep     | Arr     | Category | Price | Avail |
|-------|---------|---------|-------------|---------|---------|----------|-------|-------|
| LAXMI | BUS-105 | DHANBAD | DEOGARH     | 3:00 PM | 2:50 PM | AC       | 500   | 45    |

Enter Bus Number: BUS-105

Enter Category (AC / Non-AC / Sleeper): AC

Enter number of seats to book: 3

Booking has been completed successfully.

Total amount for your booking is: ₹1500

### BOOKING MANAGEMENT SYSTEM

- ```
=====
1. Create New Booking
2. View All Bookings
3. Delete Booking
4. Search Booking
5. View Ticket
6. Back to Main Menu
=====
```

Please enter your choice: 2

BookID	Name	Phone	BusNo	Category	Seats	Amount
BKG-1	NAYAN RAJ	9931871581	BUS-105	AC	3	1500

BOOKING MANAGEMENT SYSTEM

- ```
=====
1. Create New Booking
2. View All Bookings
3. Delete Booking
4. Search Booking
5. View Ticket
6. Back to Main Menu
=====
```

Please enter your choice: 5

Enter Booking ID to view ticket: BKG-1

### TICKET DETAILS

```
=====
Booking ID : BKG-1
Name : NAYAN RAJ
Phone : 9931871581
Bus No : BUS-105
Category : AC
Seats : 3
Amount : ₹1500
=====
```

```
Ticket No : TKT-BKG-1-1 | Seat-1
Ticket No : TKT-BKG-1-2 | Seat-2
Ticket No : TKT-BKG-1-3 | Seat-3
=====
```

➤ **MODULE 3 :- PAYMENT MODULE**

```
=====
 PAYMENT MANAGEMENT SYSTEM
=====

1. Make Payment
2. View All Payments
3. Back to Main Menu

Please enter your choice: 1
Enter Payment ID (e.g., PAY-1): PAY-1
Enter Booking ID: BKG-1

Booking Details:
Booking ID : BKG-1
Name : NAYAN RAJ
Phone : 9931871581
Bus No : BUS-105
Category : AC
Seats : 3
Total Amount Payable: ₹1500

Do you want to proceed with the payment? (y/n): y
Enter Payment Method (UPI / Card / Cash): UPI

Payment has been completed successfully.

```

```
=====
 PAYMENT MANAGEMENT SYSTEM
=====

1. Make Payment
2. View All Payments
3. Back to Main Menu

Please enter your choice: 2
PayID | BookID | Amount | Method | Status

PAY-1 | BKG-1 | 1500 | UPI | paid

```

➤ **MODULE 4:- CANCELLATION MODULE**

```
=====
 CANCELLATION MANAGEMENT SYSTEM
=====
```

- ```
1. Cancel Booking
2. View All Cancellations
3. Back to Main Menu
=====
```

```
Please enter your choice: 1
Enter Booking ID to cancel: BKG-1
Enter Cancellation ID (e.g., CAN-1): CAN-1
Enter Reason for Cancellation: unable to go
=====
```

```
Booking has been cancelled successfully.
Seats have been restored and payment (if any) removed.
=====
```

```
=====
                        CANCELLATION MANAGEMENT SYSTEM
=====
```

- ```
1. Cancel Booking
2. View All Cancellations
3. Back to Main Menu
=====
```

```
Please enter your choice: 2
CancelID | BookID | Reason | Status
=====
CAN-1 | BKG-1 | unable to go | cancelled
=====
```

## 11. LIMITATION AND FUTURE SCOPE:-

### ➤ LIMITATIONS OF THE SYSTEM:-

- **No Database Integration**  
The system uses flat text files instead of a proper database, which limits scalability and data management for large records.
- **Single User Environment**  
The system is designed for one user at a time and does not support multiple users accessing the system simultaneously.
- **Limited Security Features**  
There is no authentication or encryption, so data is not protected against unauthorized access.
- **Command-Line Interface Only**  
The system works through a terminal interface, which may not be user-friendly for non-technical users.
- **Basic Error Handling**  
The system has limited validation and may not handle all invalid inputs or edge cases.
- **No Backup or Recovery System**  
There is no automatic backup, so data loss may occur if files are deleted or corrupted

### ➤ FUTURE SCOPE:-

- **Database Integration**  
Replace flat files with a database like MySQL or SQLite for better data management and scalability.
- **Graphical User Interface (GUI)**  
Develop a user-friendly interface using web technologies or desktop applications.
- **Authentication and Security**  
Add login system, password protection, and data encryption for secure operations.
- **Online Booking System**  
Convert the system into a web-based application accessible over the internet



- **Real-Time Seat Availability**

Implement live updates for seat availability and booking status.

- **Advanced Reporting**

Generate reports such as daily bookings, revenue, and cancellation statistics.

## **12. CONCLUSION:-**

The Ticket Booking Management System developed in this project successfully demonstrates how booking operations can be efficiently managed using simple technologies like shell scripting and flat file systems. The system provides essential functionalities such as ticket booking, cancellation, viewing records, and managing customer details in a structured and organized manner.

Through this project, we understood how data can be stored, processed, and retrieved without using complex database systems, making it lightweight and easy to implement. It also helped in enhancing knowledge of file handling, scripting logic, and basic system design concepts like data flow and modular programming.

Although the system has certain limitations, such as lack of advanced security and scalability, it serves as a strong foundation for understanding real-world booking systems. Overall, this project meets its objectives and can be further improved by integrating modern technologies like databases, graphical user interfaces, and online payment systems.